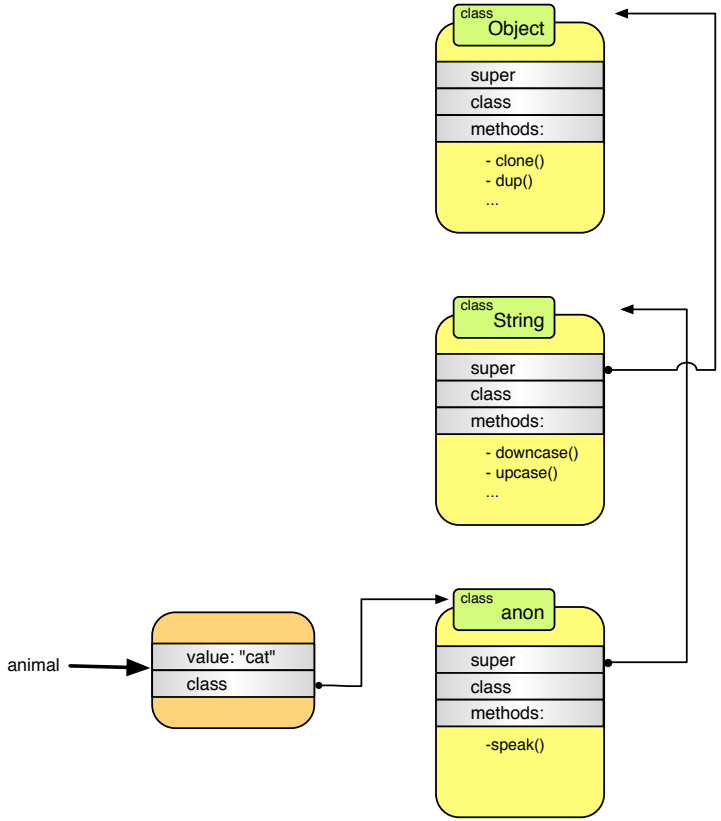
```
speak
```

If an object has a singleton class, that is the first place Ruby looks for object lookup. It is as if Ruby makes String (which was the original class of "cat") the superclass of the singleton class. The picture looks like this:



Now let's follow the call to animal.speak. Ruby goes to the object referenced by animal and then looks in its singleton class class for the method speak. The singleton class of the animal object is the newly created singleton class, and it contains the method we need.

What happens if we instead call animal.upcase? The processing starts the same way: Ruby looks for the method upcase in the singleton class but fails to find it there. It then follows the normal processing rules and starts looking up the chain of superclasses. The superclass of the singleton is String, and Ruby finds the upcase method there. Notice that there is no special-case processing here—Ruby method calls always work up the object chain in the same way.

Singletons and Classes

Earlier, we said that inside a class definition, self is set to the class object being defined. It turns out that this is the basis for one of the more elegant aspects of Ruby's object model.

Recall that we can define class methods in Ruby using either of the forms def self.xxx or (more rarely) def ClassName.xxx: